

电子科技大学信息与通信工程学院

基于 sEMG 肌电信号 手势识别控制

(实验) 课程名称 信号分析与处理

电子科技大学教务处制表

电子科技大学

基于 sEMG 肌电信号

手势识别控制

实验报告

小组名称:

名门望组

小组成员:

郝瀚宇、曹护濒、王易鸿、赵宇杰

指导老师: 武畅, 朱学勇

一、项目背景

从现实技术发展角度看,当前主流的人机交互方式仍以键盘、鼠标、触摸屏等外设为主,其本质是将人的动作或意图经过多层中间映射后输入计算系统。这种交互模式在通用计算场景中已较为成熟,但在自由度、直观性和沉浸感方面仍存在明显局限。尤其是在虚拟现实、增强现实、智能控制以及康复医疗等应用场景中,用户往往需要与系统进行连续、实时且高度自然的交互,传统外设已难以满足对实时性、自然性和连续性的更高要求。因此,构建一种能够更直接反映人体运动意图的新型人机交互系统,逐渐成为人机交互技术发展的重要研究方向。

人体肌电信号是神经系统驱动肌肉收缩时产生的生物电信号,直接反映了神经兴奋与肌肉活动之间的内在联系,能够在动作发生的早期阶段表征人体的运动意图。与基于视觉识别或机械传感的间接感知方式相比,肌电信号具有响应速度快、对外部环境依赖性低以及可直接获取生理信息等显著优势。尤其是表面肌电信号(surface EMG, sEMG),可通过非侵入式电极在体表完成采集,在保证安全性与佩戴舒适性的同时,为实时人机交互系统的构建提供了可行且稳定的技术基础。因此,基于肌电信号的人机交互方式被认为是一种更贴近人体本能、具有广阔应用前景的新型交互形式。

基于上述背景,本项目旨在设计并实现一套非侵入式手臂肌电信号采集与处理系统,通过对手臂关键肌群肌电信号的实时采集与分析,完成肌电信号的预处理、特征提取及手势识别验证,最终实现基于手势的实时交互控制。该研究不仅有助于加深对肌电信号特性及其工程实现方法的理解,也为肌电信号在人机交互、智能控制及虚拟现实等领域的进一步应用提供实验依据和技术参考,具有一定的工程实践意义与应用价值。

二、理论验证

基于相关文献，本实验首先验证了多通道表面肌电信号（sEMG）用于手势识别的可行性。研究表明，sEMG 信号在肌肉收缩初始阶段具有确定性结构，可通过模式识别方法区分不同运动意图。例如，早期工作利用时间域特征（如均值绝对值、波形长度等）和人工神经网络分类器，成功实现了多类肢体运动的高精度识别，准确率最高可达 98%^[1]。进一步地，多通道 sEMG 方案通过提取统计特征（如均值绝对值和过零率）并结合线性判别分析，在 12 名受试者中实现了超过 90% 的平均准确率，证明多通道信号能提供局部化信息，增强类间分离度^[2]。这些结论共同表明，多通道 sEMG 结合模式识别技术是一种可靠的手势判别方案。

其次，频率域特征在 sEMG 分析中的有效性得到近期文献支持。例如，Zhang 等^[3]提出的双通道注意力融合网络（DCAF-Net）从双侧腓肠肌 sEMG 信号中提取了 48 个时间域和频率域特征，用于预测下肢运动意图。特征列表包括平均功率密度（Mean Power Spectral Density, MPSD）、频率重心方差（Frequency Center Variance, FCV）、平均频率（Mean Frequency, MNF）等多种指标，实验在 8 名卒中患者中实现了 97.19% 的预测准确率。该研究强调，频率特征能稳定编码运动意图的频域信息，且对噪声鲁棒性强，为将频率特征应用于手势识别提供了直接依据。

在综合分析上述文献后，结合项目时间限制，我们先计算一组频率域特征：平均功率密度（MPSD）、频率重心（FCV）、平均频率（MNF）、频率均方根（Root Mean Square Frequency, RMSF）和频率标准差（Standard Deviation of Frequency, SDF）。最终，基于特征的有效性和计算简便性，我们选取平均功率密度（MPSD）作为手势判断的主要依据，其余特征可用作后续算法升级时的可用选项。

三、系统架构

1. 信号采集模块

硬件：Muscle Sensor v3

由于经费限制，本项目采用两个 Muscle Sensor v3 肌电传感器构成双通道采集系统，以提升信号的空间分辨率和准确性。传感器通过模拟输出引脚（SIG）将采集到的表面肌电信号（sEMG）输入到 STM32H750 的 ADC 引脚（PA8, PC6）。代码中配置了双同步 ADC 模式（ADC1 和 ADC2），以 2000 Hz 的采样率同步采集两路信号，每帧 256 点（对应 128 ms 时长）。同时，ADC3 用于监测内部参考电压（VREFINT），以校准采样值，减少电源波动引入的误差。

2. 信号处理模块

硬件：STM32H750 核心开发板

本系统选用 STM32H750 微控制器作为处理核心，其优势包括：

小型化设计：核心板尺寸紧凑，适合可穿戴设备集成。

高性能算力：基于 Cortex-M7 内核，主频可达 480 MHz，支持单精度浮点运算（FPU），能实时处理复杂的信号处理算法。

丰富外设：内置多路 ADC、DMA 控制器和定时器，可高效实现数据采集和流水线处理，确保系统实时性。


```
float iir_sos_coeffs[IIR_STAGES * 5] = {
    1.0f, 0.0f, -1.0f, 0.8693764321864266f, -0.9536568122771949f,
    1.0f, 0.0f, -1.0f, 1.9864247945818152f, -0.9950821659378192f,
    1.0f, 0.0f, -1.0f, 0.8321927690588344f, -0.8672413716423795f,
    1.0f, 0.0f, -1.0f, 1.9766845753697753f, -0.9853141721013935f,
    1.0f, 0.0f, -1.0f, 0.7995116982474844f, -0.7884588794031837f,
    1.0f, 0.0f, -1.0f, 1.9670113874654054f, -0.9756280179750199f,
    1.0f, 0.0f, -1.0f, 0.770848011898446f, -0.716535449340347f,
    1.0f, 0.0f, -1.0f, 1.957395916219848f, -0.9660142440349393f,
```

```
const float iir_scale_vals[IIR_STAGES] = {
    0.4804150143471112f, 0.4804150143471112f, 0.4686281482944881f,
    0.4686281482944881f, 0.4577285824460943f, 0.4577285824460943f,
    0.4476494583907268f, 0.4476494583907268f, 0.4383307501573455f,
    0.4383307501573455f, 0.4297185698995845f, 0.4297185698995845f,
    0.421764541012795f, 0.421764541012795f, 0.4144252363555915f,
    0.4144252363555915f, 0.407661677559951f, 0.407661677559951f,
    0.401438890604339f, 0.401438890604339f, 0.3957255125638296f,
    0.3957255125638296f, 0.3904934445236531f, 0.3904934445236531f,
```

```
// IIR滤波
```

```
arm_biquad_cascade_df1_f32(&iir_filter1, emg1_buf, emg1_filtered, FFT_POINT);
arm_biquad_cascade_df1_f32(&iir_filter2, emg2_buf, emg2_filtered, FFT_POINT);
```

```
// 预处理后的IIR系数（包含增益），格式：[b0*g, b1*g, b2*g, a1, a2]
float scaled_iir_coeffs[IIR_STAGES * 5];
```

2.1.3 50Hz 陷波滤波器

陷波滤波器主要用于消除 50Hz/60Hz 工频干扰，提高肌电信号质量。

代码中配置了单二阶节陷波滤波器，其传递函数在 50 Hz 处形成深度衰减。系数 notch_coeffs 通过 MATLAB 设计，结构同为直接 II 型。滤波在 IIR 滤波之后执行，进一步净化信号，确保 FFT 分析的准确性。

再对信号进行增强（30dB）

```

// 陷波滤波器系数（MATLAB生成的直接II型滤波器）
// 单个二阶节，用于FFT前的信号处理
#define NOTCH_STAGES 1 // 陷波滤波器二阶节数量
const float notch_coeffs[NOTCH_STAGES * 5] = {
    // b0, b1, b2, a1, a2
    0.999334363637892630904957513848785310984f,
    -1.974061798642416265536780883849132806063f,
    0.999334363637892630904957513848785310984f,
    1.974061798642416265536780883849132806063f, // 注意：MATLAB的a1取反
    -0.998668727275785261809915027697570621967f // 注意：MATLAB的a2取反
};

```

```

// 陷波滤波（在IIR滤波之后，FFT之前）
iir_filter_frame(emg1_filtered, emg1_notch_filtered, FFT_POINT,
                &notch_filter1);
iir_filter_frame(emg2_filtered, emg2_notch_filtered, FFT_POINT,
                &notch_filter2);

```

2.2 特征提取

特征提取将时域信号转换为频域特征，以量化肌肉活动的能量分布。

在该部分中，单片机先对预处理后的信号进行 FFT 运算，在通过运算后的结果求解出 MPSD 用于手势判断。

2.2.1 FFT 变换

快速傅里叶变换（FFT）将 256 点时域信号转换为频域表示。代码中函数 `calc_fft` 执行以下步骤：

加窗：应用汉明窗（`hamming_window`）减少频谱泄漏，窗函数系数通过 $0.54 - 0.46 * \cos(2\pi n/N)$ 生成。

```

void hamming_window_init(void) {
    // 生成汉明窗系数: w[n] = 0.54 - 0.46 * cos(2π * n / (N - 1))
    for (uint16_t n = 0; n < FFT_POINT; n++) {
        float angle = 2.0f * PAI * n / (FFT_POINT - 1);
        hamming_window[n] = 0.54f - 0.46f * arm_cos_f32(angle);
    }
}

```

FFT 计算：调用 CMSIS-DSP 库的 `arm_cfft_f32` 进行复数 FFT，输出实部与虚部。

幅值计算：通过 `arm_cmplx_mag_f32` 得到幅度谱，并乘以归一化因子补偿窗函数能量损失，为进一步的频域分析打好基础。

```

void calc_fft(float *sample_buf, float *fft_in, float *fft_mag, uint16_t N) {
    // 构造FFT输入并应用汉明窗
    for (uint16_t n = 0; n < N; n++) {
        // 应用汉明窗以减少频谱泄漏
        fft_in[2 * n] = sample_buf[n] * hamming_window[n];
        fft_in[2 * n + 1] = 0.0f;
    }
    // 初始化并执行FFT
    arm_cfft_instance_f32 *fft_inst;
    switch (N) {
        case 128:
            fft_inst = &arm_cfft_sR_f32_len128;
            break;
        case 64:
            fft_inst = &arm_cfft_sR_f32_len64;
            break;
        case 256:
            fft_inst = &arm_cfft_sR_f32_len256;
            break;
        default:
            while (1)
                ; // 非法点数，卡死报错
    }
}

arm_cfft_f32(fft_inst, fft_in, 0, 1);
// 计算幅值
arm_cmplx_mag_f32(fft_in, fft_mag, N);

// 应用汉明窗归一化因子，补偿窗函数引起的能量损失
for (uint16_t k = 0; k < N; k++) {
    fft_mag[k] *= hamming_norm;
}
}

```

2.2.2 计算 MPSD

平均功率谱密度（MPSD）是肌电信号在有效频段内的能量均值。函数 `extract_feature` 计算流程如下：

频段索引映射：根据采样率和 FFT 点数，确定有效频率对应的索引范围。

功率累加：对每个频率索引 k ，功率值为幅值的平方，累加有效频段内的总功率。

归一化：总功率除以频段点数，得到 MPSD。该特征反映肌肉收缩的整体能量水平。

```

// 提取有效频段的功率谱密度（PSD）特征值
// 功率谱 = 幅值平方，反映频段能量
float extract_feature(float *fft_mag, uint16_t N, float fs, float f_min,
                    float f_max) {
    float sum_power = 0.0f; // 累加功率而非幅值
    // 计算有效频率对应的FFT索引范围
    uint16_t k_min = (uint16_t)(f_min * N / fs);
    uint16_t k_max = (uint16_t)(f_max * N / fs);
    // 边界保护（避免索引越界）
    k_max = (k_max >= N) ? (N - 1) : k_max;
    // 累加有效频段的功率谱密度（幅值平方）
    for (uint16_t k = k_min; k <= k_max; k++) {
        // 功率 = 幅值^2，归一化可选：/N 或 /(N^2)
        float power = fft_mag[k] * fft_mag[k];
        sum_power += power;
    }
    // 返回平均功率密度（可选归一化）
    return sum_power / (float)(k_max - k_min + 1);
}

```

2.3 手势判断

手势识别基于双通道 MPSD 特征的阈值判断。函数 `recognize_gesture` 实现以下逻辑：

特征平滑：连续 5 帧特征值（`FEATURE_BUF = 5`）存入环形缓冲区，计算移动平均值以抑制波动。

阈值比较：将平滑后的特征值（`feat1` 和 `feat2`）与预设阈值（`FEATURE1 = 20`, `FEATURE2 = 20`）比较，判定手势：

- `GESTURE_FIST`（握拳）：双通道特征均大于阈值。
- `GESTURE_RIGHT`：通道 1 特征大于阈值，通道 2 小于阈值。
- `GESTURE_LEFT`：通道 1 特征小于阈值，通道 2 大于阈值。
- `GESTURE_NONE`：双通道特征均小于阈值。

实时输出：识别结果通过 UART2 发送数字编码（0-3），供上位机解析。
阈值的设定通过我们多次测试得到。

```

Gesture_TypeDef recognize_gesture(float feat1, float feat2) {
    static int i=0;
    feature_buf1[i]=feat1;
    feature_buf2[i]=feat2;
    i++;
    if((i+1)%FEATURE_BUF==0)
    {
        float sum1=0,sum2=0;
        for(int j=0;j<=i;j++)
        {
            sum1+=feature_buf1[j];
            sum2+=feature_buf2[j];
        }
        sum1=sum1/FEATURE_BUF;
        sum2=sum2/FEATURE_BUF;
        if(sum1>FEATURE1&&sum2>FEATURE2)
        {
            current_gesture=GESTURE_FIST;
        }
        if(sum1>FEATURE1&&sum2<FEATURE2)
        {
            current_gesture=GESTURE_RIGHT;
        }
        if(sum1<FEATURE1&&sum2>FEATURE2)
        {
            current_gesture=GESTURE_LEFT;
        }
        if(sum1<FEATURE1&&sum2<FEATURE2)
        {
            current_gesture=GESTURE_NONE;
        }
        i=0;
    }
    return current_gesture;
}

```

3.信号发送模块

硬件:ESP-01

最常用的 WIFI 模块，其吞吐量足够在满足实时性的情况下将手势的判断结果发送给电脑。通过 STM32H750 的 USART2 串口将数据输入到 ESP-01 模块中完成传输

4.UI 展示模块

将接收到的手势判断结果以柱状图的形式进行展示，并且最终汇总出统一的手势结果，并能根据该结果控制其内置的雷霆战机小游戏中飞机的移动。

要点：由于 WIFI 模块发送的是最原始的判断结果，存在抖动现象，因此为了增强手势判断结果的稳定性，应当进行消抖的处理

```
function processData(rawData) {
    rawData = rawData.trim();
    if (rawData === "esp_connected") {
        wifiStatusDot.classList.add("connected");
        wifiStatusText.textContent = "WiFi模块已连接";
        logData(`[${formatTimeWithMs(new Date())}] WiFi模块已连接`);
        return;
    } else if (rawData === "esp_disconnected") {
        wifiStatusDot.classList.remove("connected");
        wifiStatusText.textContent = "WiFi模块已断开";
        logData(`[${formatTimeWithMs(new Date())}] WiFi模块已断开`);
        return;
    }

    if (rawData === "disconnect") {
        disconnectHandler();
        return;
    }

    const stateNum = parseInt(rawData);
```

```
if (isNaN(stateNum)) {
  logData(
    `[${formatTimeWithMs(
      new Date()
    )}] 数据格式错误: ${rowData} 应为0、1、2、3`
  );
  return;
}

// 将有效状态数据写入环形缓冲区，供图表统计使用
writeToRingBuffer(stateNum);

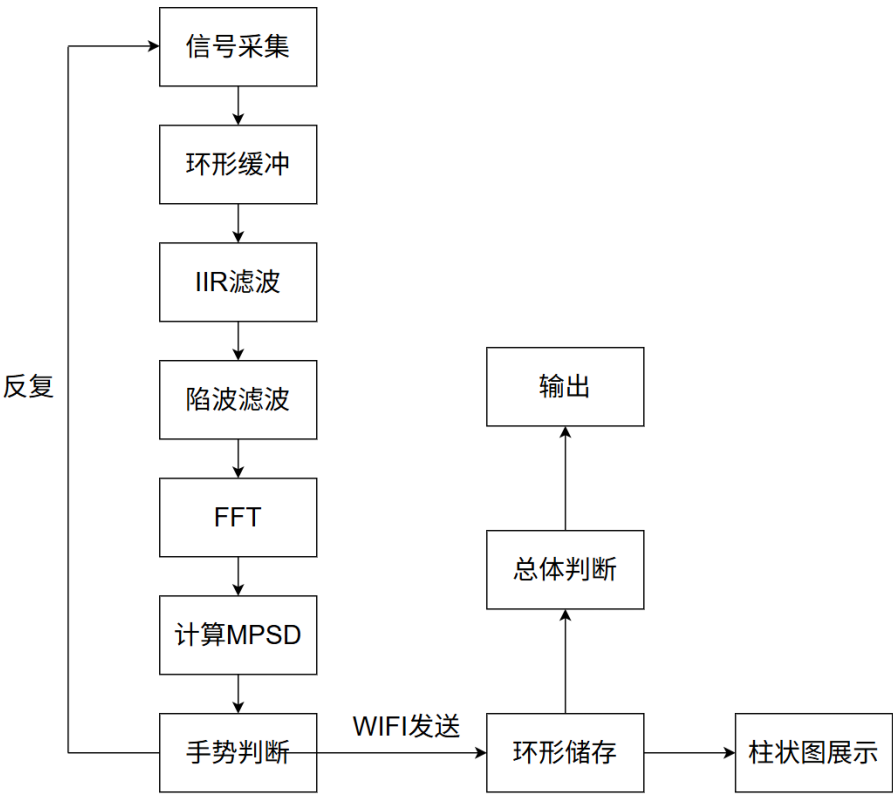
const stateName = stateMap[stateNum]?.name || "未知状态";
const displayText = `[${formatTimeWithMs(
  new Date()
)]] 识别状态: ${stateName} (${stateNum})`;
logData(displayText);

handlePlayerMove(stateNum);

// 图表会在刷新时实时统计环形缓冲区数据，无需累积

// 使用节流函数替代直接刷新图表
scheduleChartUpdate();
}
```

四、系统框图



五、测试

- 1. 测试环境与条件：室内恒温 27° 左右，被试者身体健康，左臂活动正常自如。
- 2. 测试对象：3 人，分别为郝瀚宇，赵宇杰，王易鸿
- 3. 测试动作：
 - 1) 静息：手掌自然平放，五指舒展
 - 2) 握拳：五指握紧成拳
 - 3) 左偏：手掌整体向左倾斜（偏转角度约 30°-90°）
 - 4) 右偏：手掌整体向右倾斜（偏转角度约 30°-90°）
- 4. 评价指标

单动作正确率 = (正确识别次数 / 单项测试次数) × 100%
整体平均正确率 = (所有动作正确识别次数总和 / 总测试次数) × 100%

5. 测试结果：

动作类别	测试次数	正确识别次数	正确率
静息	300	300	100%
左偏	300	283	94.3%
右偏	300	276	92.0%
握拳	300	246	82.0%

综合正确率：92.0%

6. 结论：

- 1) 静息姿态识别效果最优，未出现误判。
- 2) 左偏与右偏动作识别正确率相近（92% - 94%），可能存在轻微方向混淆。
- 3) 握拳动作识别率相对较低（82%），需进一步分析误判样本（如与弯曲手指姿态的相似性干扰）。

六、问题与改进方案

1. 阈值确定

目前的阈值判定是预先输入的固定值，但因为每个人的实际情况均不同，因此不同人的阈值也应该是动态变化的。因此可以通过计算功率谱的二阶中心矩对阈值进行进一步确认后再进行手势判断，产品的泛用性就会增强。

因时间有限未调用的二阶中心矩计算函数：

```
float calc_freq_standard_deviation(float *fft_mag, uint16_t N, float fs, float f_min, float f_max)
{
    // 1. 先计算重心频率 (FCG)
    float fcg = calc_frequency_cg(fft_mag, N, fs, f_min, f_max);
    if (fcg < 1e-6f) // FCG计算失败时返回0
    {
        return 0.0f;
    }

    // 2. 计算有效频率对应的FFT索引范围
    uint16_t k_min = (uint16_t)(f_min * N / fs);
    uint16_t k_max = (uint16_t)(f_max * N / fs);
    // 边界保护: 避免索引越界
    k_min = k_min < 0 ? 0 : k_min;
    k_max = k_max >= N ? (N - 1) : k_max;
    if (k_min > k_max) return 0.0f;

    // 3. 累加功率和、(f_k - FCG)^2-功率乘积和
    float sum_power = 0.0f; // 分母: ΣP(k)
    float sum_freq_diff2_power = 0.0f; // 分子: Σ((f_k - FCG)^2 * P(k))
    for (uint16_t k = k_min; k <= k_max; k++)
    {
        float f_k = (float)k * fs / N; // 当前索引对应的实际频率
        float p_k = fft_mag[k] * fft_mag[k]; // 功率谱密度P(k)=|FFT(k)|^2
        float freq_diff = f_k - fcg;
        sum_power += p_k;
        sum_freq_diff2_power += (freq_diff * freq_diff) * p_k;
    }

    // 4. 避免除以0错误
    if (sum_power < 1e-6f)
    {
        return 0.0f;
    }

    // 5. 计算频率标准差
    float fsd = sum_freq_diff2_power / sum_power;
    return sqrt(fsd); // 使用CMSIS-DSP的平方根函数(更高效)
}
```

2.特征分析

大部分文献中的特征分析均是结合了时域、频域多种特征综合分析，得到的手势识别结果的准确率较高。为了提高我们产品的准确性，应当提取更多的特征进行综合分析。

因时间有限未调用的频谱特征提取函数：

特征名称	函数名
重心频率（FCG）	calc_frequency_cg
平均频率（MF）	calc_mean_frequency
频率均方根（RMSF）	calc_rms_frequency
频率标准差（FSD）	calc_freq_standard_deviation

（因为代码量太大，此处暂时不附加代码）

七、项目收获

1. 完成情况总结

本项目围绕“手臂表面肌电信号(sEMG)的采集与处理及其在手势识别中的应用”这一目标，完成了从信号采集、嵌入式实时处理到结果传输与上位机展示的系统闭环搭建与验证。在硬件层面，采用双通道 Muscle Sensor v3 实现手臂 sEMG 的非侵入式采集，并将模拟信号接入 STM32H750 的 ADC 进行同步采样；在软件与算法层面，完成了基于 DMA 环形缓冲区的连续采集机制，构建了包含 IIR 滤波与 50 Hz 陷波在内的预处理流程，并在单片机端实现加窗 FFT 与频域特征提取，以平均功率谱密度(MPSD)作为主要特征完成阈值判别；在系统联调层面，通过串口与 ESP-01 模块实现识别结果的无线发送，并在上位机 UI 中以可视化方式展示识别输出并进行抖动汇总，最终实现了对交互控制场景（小游戏飞机移动）的功能验证。总体而言，本项目在经费与周期约束下实现了预期的核心功能链路，验证了双通道 sEMG 结合轻量级频域特征与阈值策略进行手势识别的工程可行性，同时也在实验过程中明确了阈值个体差异与结果抖动等后续可改进方向。

2. 理论应用到工程实践的差距

通过将《信号与系统》《数字信号处理》等课程中学习的采样理论、滤波器设计、频谱分析与特征提取方法应用到实际系统中，我们对“理论可行”与“工程可用”之间的差距有了更直观的认识。课堂推导往往基于相对理想的信号模型与可控的噪声假设，而真实的 sEMG 采集环境同时受到电极贴合状态变化、皮肤阻抗波动、运动伪迹、工频耦合以及供电扰动等多种因素影响，导致信号噪声成分更复杂且时变性更强，因此预处理策略不仅需要满足频带需求，还必须兼顾数值稳定性、实时性与对异常情况的鲁棒性。与此同时，课堂中的算法实现常以离线处理为主，计算资源与时间相对充裕，而本项目在 STM32H750 上进行实时处理时，需要严格受限于采样率、帧长与中断调度带来的时间预算约束，任何滤波阶数、窗函数选择、FFT 计算与特征统计的细节都会直接影响系统能否稳定实时运行。此外，理论上某一特征能够表征信号能量或频谱分布，并不必然意味着它在不同个体、不同佩戴位置或不同肌肉状态下具有一致的判别边界，这也使我们认识到工程系统往往需要引入校准、阈值自适应或多特征融合等机制来提升泛化能力与稳定性。本次实践使我们对课程知识的理解从“公式与方法”进一步深化为“在约束条件下做取舍并保证系统可运行”的工程思维。

3. 硬件与软件配合及跨板块沟通的重要性

本项目属于典型的多模块强耦合系统，团队协作效率直接决定最终系统效果。在实施过程中我们体会到，硬件端的设计思路应当与软件端相配合，尽可能的采集到更多、更准确、更有效的信息；嵌入式软件端的采样同步、DMA 缓冲与中断调度决定了数据是否连续可靠，若采集与处理节奏不匹配，会引入数据覆盖或延迟累积，从而影响特征统计的一致性；上位机端的可视化与抖动汇总决定了最终交互体验，若缺乏与下位机输出时序、编码规则的一致约定，就会在系统联调阶段出现“数据能传但难以稳定使用”的问题。项目推进过程中，通过明

确接口规范、及时同步各模块状态、在联调中快速定位问题来源并形成闭环修改，我们认识到积极沟通不仅能减少返工成本，更能避免各模块各自优化却整体效果下降的情况。该过程提升了我们对系统工程协作模式的理解，也强化了在多学科交叉项目中以整体目标为导向、以数据与结果为依据进行协同迭代的能力。

八、参考文献

- [1] B. Hudgins, P. Parker, and R. N. Scott, “A new strategy for multifunction myoelectric control,” *IEEE Transactions on Biomedical Engineering*, vol. 40, no. 1, pp. 82–94, Jan. 1993.
- [2] K. Englehart and B. Hudgins, “A robust, real-time control scheme for multifunction myoelectric control,” *IEEE Transactions on Biomedical Engineering*, vol. 50, no. 7, pp. 848–854, Jul. 2003, doi: 10.1109/TBME.2003.813539.
- [3] L. Zhang, Y. Liu, H. Liu, Z. Sun, H. Zhang, Y. Zhang, and X. Ma, “DCAF-Net: Dual-Channel Attentive Fusion Network for Lower Limb Motion Intention Prediction in Stroke Rehabilitation Exoskeletons,” *arXiv preprint arXiv:2512.12184v1*, Dec. 13, 2025.